

rl-matdesign — YAML Configuration Reference

Every config flag, what it does, and its choices

2026-06-10

Overview

A run is configured by one YAML file plus a few CLI flags. The three pluggable layers are chosen by **env_type** (the search space), **predictor** (the reward), and **constraint_filter** (action masking); each then reads its own keys. The RL and generation knobs are shared across scenarios.

- The YAML is the base; a few CLI flags override matching keys.
- **Seeds are CLI-only:** `--dp-seed` (predictor RNG), `--train-seed` (training; defaults to `--dp-seed`), `--gen-seed` (generation; defaults to `--dp-seed`).
- The full merged config is written to `<out>/run_config.json`.

CLI flags: `--config` (required), `--out` (required), `--method`, `--dp-seed`, `--train-seed`, `--gen-seed`, `--device` (cpu/cuda), `--dqn-loss`, `--dqn-augment-permutations`, `--save-checkpoint-freq`, `--resume-training`, `--only-generate`, `--skip-generation`, `--max-gen-attempts`, `--load-qnet` / `--load-policy` / `--load-value-net` / `--load-scaler`.

1. Environment

Key	What it does	Choices / default
env_type	Selects the environment	<code>fraction</code> (default, <code>CompositionEnv</code>); <code>integer_ratio</code> (<code>IntegerRatioEnv</code>); <code>multi_group</code> (<code>MultiGroupEnv</code>)

fraction env keys

Key	What it does	Choices / default
cation_set	Candidate element symbols	required (list)
fraction_set	Allowed fraction strings (grid)	default = built-in <code>0.05 ... 0.80</code>
total_units	Grid resolution; step = $1/\text{total_units}$	<code>20</code> ($\rightarrow 0.05$); use <code>100</code> for <code>0.01</code>
n_components	Distinct elements per episode	<code>5</code>
anion_formula	Fixed suffix appended to the formula	<code>""</code> (e.g. <code>02H1</code> , <code>03</code>)
episode_style	How each step picks	<code>element_then_amount</code> (default: choose element + amount); <code>fixed_order_amount</code> (order pinned to <code>cation_set</code> , choose only amounts)
element_bounds	Per-element <code>{el: [min,max]}</code> fraction caps	<code>none</code> ; only with <code>fixed_order_amount</code>
constraint_filter	Action-masking filter (see §3)	<code>null</code>

integer_ratio env keys

Key	What it does	Choices / default
cation_set	Candidate elements	required
ratio_set	Allowed integer-ratio digits	default <code>"0" ... "9"</code>
n_components	Elements per episode	<code>5</code>
constraint_filter	Action-masking filter	<code>null</code>

No sum-to-1 constraint; the composition is normalized afterward.

2. multi_group env — groups:

groups: is an **ordered list** of sublattice **groups**, filled in order. A group is the atomic unit; a single fraction/integer_ratio env is the one-group case. The predictor receives the structured {group_name: {element: value}}.

Common group keys

Key	What it does	Choices / default
name	Group label (appears in the structured terminal)	string
kind	Group type	composition (default) or categorical
sites	Sublattice size (atoms per formula unit) — bridges fraction<->count and assembles the chemical formula (amount × sites)	1
constraint_filter + its kwargs	Per-group filter (sees prior_groups for cross-group coupling)	(see §3)

kind: composition — pick N elements with amounts summing to 1. Reuses the fraction-env keys (cation_set, fraction_set, total_units, n_components, episode_style, element_bounds) plus friendly knobs:

Key	What it does	Choices / default
amount	Generate the value grid instead of writing fraction_set	{min, max, step} or a list
host	A host element auto-takes the leftover (list only the dopants; wires a host_complement filter)	element symbol

kind: categorical — pick discrete **real values**; no sum-to-1. Each slot is one element with its own value list; the terminal returns the chosen values unchanged (so a builder reads $Cl = 1.0 / 0 = 1$ directly).

Key	What it does
choices	[{element, values: [...]}, ...] — one slot per element

A categorical filter may mask a slot by an earlier group (e.g. sse_doping masks the O slot by the P-site metal's category).

3. Constraint filters (constraint_filter:)

Value	What it does	Its keys
null	No constraint (default)	—
smact_charge	Keep only compositions that can be charge-neutral (SMACT oxidation states)	smact_anions: [{symbol, charge, stoich}] (or smact_anion / smact_anion_charge / smact_anion_stoich)

Value	What it does	Its keys
last_step_element	Force the last step to a required element	required_elements; nonzero_ratio_at_last (true); reserve_for_last (true)
phase_pattern	Keep compositions matching phase patterns	phase_patterns
ooh_phase chain	OOH oxyhydroxide phase screen Apply several filters in sequence	target_phases (default [any]) filters: [{constraint_filter: ..., ...}, ...]
host_complement	Dopants at a level, host takes the rest (wired automatically by a composition group's host knob)	host_element, levels
sse_doping	Mask the LiPS S-site O-form slot by the metal's category	o_element (O), host_P (P), metal_only, oxide_only
"pkg.mod:Class"	Your own filter (FQN)	your class's keys

4. Predictors (predictor:)

Value	What it does
dummy	Random reward (default; no models — smoke tests)
structure_score	The one structure-based predictor — build -> [relax] -> score N properties -> combine (see below)
ooh	OOH overpotential (adsorbate slabs)
sinter_calcline	RandomForest sintering/calcline temperature
"pkg.mod:Class"	Your own predictor (FQN)

structure_score keys

One predictor steered by dials. The pipeline is `builder.build -> [relax once, optional] -> score each property -> combine`. It replaces the former `dp_structure / dp_property / composite / structure_pipeline` (and the `hea / perovskite` aliases): each is now just a particular setting of the dials below.

Key	What it does	Default
builder	Builder name/FQN (substitute = fixed-lattice element swap; sse = doped supercell; or <code>pkg.mod:Class</code>)	substitute
share_structure	true = build+relax one cell, score all properties on it; false = each property builds its own	true
n_random_configs	Random placements per candidate	1
geo_opt	Relaxation stage (see below); omit / enabled: false to skip	optional
properties	Non-empty list of objective specs (see below)	required
k	Uncertainty coefficient	1.0
sweep	Optional {name, values} — optimize a shared operating condition (e.g. temperature) per composition (see below)	optional
base_poscar / site_symbol	Builder knobs (for substitute); inherited by per-objective builders when <code>share_structure: false</code>	—

`geo_opt`: sub-keys — model (default `models/DPA-3.1-3M.pt`), head (user-defined), `fmax` (0.001), steps (1000), `relax_cell` (true), `enabled` (true).

`properties[]`: sub-keys:

Key	What it does	Default
name	Unique label (CSV column prefix)	<code>prop{i}</code>
backend	energy (DP potential energy via ASE) or property (DP DeepProperty head)	property
models (or legacy <code>dp_models</code>)	Ensemble checkpoint list	required
head	DP head (energy: calculator head; property: DeepProperty head)	none
direction	max / min (energy: use min for “lower is better”)	max

Key	What it does	Default
weight / scale objective	Combine weight / unit scale mean_minus_kstd / mean / mean_plus_kstd	1.0 / 1.0 mean_minus_kstd
energy_per_atom (<i>energy backend</i>)	Normalize by atom count	true
output_index / output_aggregator (<i>property backend</i>)	Which vector component / collapse mode	0 / index (index/mean/max)
fparam / aparam (<i>property backend</i>)	Frame / atomic parameters for heads trained with them. A null in fparam marks the slot filled by the sweep value (requires a top-level sweep)	none

Reward = sum weight * objective_from_mean_std(direction*mean, std, objective, k) / scale over the properties (identical formula in both share_structure regimes).

sweep: — per-composition operating-condition optimization

Some property heads depend on an external operating condition (e.g. temperature) passed as a frame parameter. sweep: {name, values} turns that condition into an **inner optimization**: for each composition the predictor scores every property at each value and keeps the single shared value maximizing the **combined** reward. Because the structure is built + relaxed **once** and reused, sweeping is cheap (only the property inference repeats).

```
sweep:
  name: temperature
  values: [460, 470, 480, 490]
properties:
  - name: conductivity
    backend: property
    fparam: [3.9568e-05, null, 6]      # null = the swept temperature slot
    direction: max
    ...
  - name: stability
    backend: property
    fparam: [3.9568e-05, null, 6, 4]
    direction: max
    ...
```

The chosen value is logged per candidate as obj_<name>_mean (e.g. obj_temperature_mean) in generated.csv. A property with no null in its fparam (or an energy backend) is condition-independent — scored once and contributing equally at every sweep value.

Migration cheatsheet (old -> structure_score):

Old predictor	Now
dp_structure	builder: substitute, one property backend: energy, direction: min
dp_property	builder: substitute, one property backend: property
hea / perovskite structure_pipeline	as dp_structure, with site_symbol: X / Fe share_structure: true (default), N backend: property objectives
composite	share_structure: false, the objectives: list becomes properties:

ooh keys

base_poscar, dp_models, objective, k, n_random_configs, ads_height, ads_dz, geo_opt (bool), geo_opt_model, uncertainty (models/configs/total), output_index, target_phases.

sinter_calcine keys

rf_model (joblib path), mode (sinter / calcine).

5. Builders (builder:)

A builder turns the agent's pick (a flat {element: fraction} or a structured {group: {...}}) into ASE structures. Built-ins:

builder:	What it does
substitute	Fixed-lattice element swap onto site_symbol placeholder sites (no vacancies). Keys: base_poscar (or poscar), site_symbol (default X).
sse	Doped-supercell recipe (P->metal, S->O/Cl/Br, charge-neutral Li vacancies). Keys below.
"pkg.mod:Class"	Your own builder (FQN) with build(candidate, *, n_configs, rng).

sse builder keys

Key	What it does	Default
base_poscar	Base supercell	required
valences	Charge table {el: int \ {sulfide, oxide}} (drives the Li-vacancy solve and oxide O count)	required
halide_total	Cl + Br per formula unit	1.7
eligible_region	Substitutable S region	{symbol: S, take: last, count: 1000}
formula_units	Formula units in the supercell — inferred from the POSCAR (host-P count / p_site_per_fu) unless set	inferred
p_site_group / s_site_group	Group names to read	P_site / S_site
host	{P, S, Li} host symbols	{P:P, S:S, Li:Li}
p_site_per_fu / s_site_per_fu / li_per_fu	Sites per formula unit	1 / 6 / 6

The S-site reads the categorical values directly — O is a form flag (0 metal / >0 oxide), Cl is a real per-f.u. count. Br = halide_total - Cl and the charge-neutral Li vacancy are derived. (No cl_map / o_off.)

6. RL method & hyperparameters

Key	What it does	Choices / default
method	Algorithm (CLI --method overrides)	a2c (default), reinforce, dqn

Policy gradient (a2c / reinforce)

Key	Default	Key	Default
pg_warmup_eps	200	pg_entropy_coef	0.01
pg_num_iters	1000	pg_repeat_penalty_coef	0.0
pg_batch_eps	15	pg_repeat_penalty_shapelog (also sqrt)	
pg_lr_actor	0.001	gamma (or pg_gamma)	0.9
pg_lr_critic	0.001 (a2c only)		

DQN

Key	Default	Key	Default
dqn_lr	0.001	dqn_target_update_freq	100
dqn_hidden_dim	256	dqn_eps_anneal_eps	10000
dqn_batch_size	256	dqn_eps_min	0.05
dqn_warmup_eps	->pg_warmup_eps/500	dqn_gamma (or gamma)	0.9
dqn_num_train_eps	20000	dqn_loss	smoothl1 (also mse)
dqn_buffer_size	50000	dqn_augment_permutation	0
dqn_grad_steps_per_ep	5		

Generation (all methods)

Key	What it does	Default
num_gen_eps	Final candidates to generate	200
exploration_gen_eps	Extra high-temperature candidates	0
gen_temperature	Sampling temperature (DQN Boltzmann / PG)	1.0
gen_top_frac	Restrict sampling to top fraction of actions	0.0
gen_epsilon	ϵ -random during generation	0.0
k	Uncertainty coefficient for reward folding	1.0